

Archive de session — 27 septembre 2026

Journée en trois temps : documentation des Vagues V1-V3 du plan de mutualisation skills/MCP (suivi de session), reprise et clôture du chantier MCP OmniRoute (validation end-to-end, service systemd), puis capitalisation en mémoire d'un piège méthodologique découvert pendant la génération de cette archive elle-même. Les trois sessions sont regroupées ici sous la même date, sections numérotées en continu.

Partie A — Documentation des Vagues V1-V3 (plan mutualisation skills/MCP)

Objectif : intégrer dans le suivi de session du projet (`JOURNAL.md/Handoff.md`) le travail des Vagues V1 à V3 du plan de mutualisation skills/MCP, déjà engagé sur `Riemann_Lab_IA` lors d'une session précédente — sans toucher au cœur de calcul ni lancer les vagues suivantes.

1. Consigne initiale

Instruction transmise en une seule fois, citée verbatim (extraits) :

Contexte : je continue le travail sur les skills de `Riemann_Lab` (branche `Riemann_Lab_IA`). Voici ce qui a été fait aujourd'hui (27/09/2026), à intégrer dans le suivi de session du projet (`JOURNAL.md` et `Handoff.md` du wiki, selon les conventions déjà en place — vérifie d'abord si `~/riemann_handoff/Handoff.md` (local, hors dépôt) est toujours la copie active la plus à jour [...], et mets à jour la bonne copie en premier ; propage ensuite si besoin, sans dupliquer).

Le message détaillait le contenu des Vagues V1-V3 (déjà engagé), les décisions ouvertes et les vagues restantes (V4, V5), et fixait explicitement le périmètre :

Ne touche à rien d'autre (pas de Vague V4/V5 sans validation explicite de ma part), et ne modifie jamais le cœur de calcul (`compute_zeros_v*.py`, `illinois_*`, `scan_arb*`) dans cette tâche de documentation.

2. Vérification de l'état réel du dépôt

`git status` et `git log --oneline -8` exécutés sur `Riemann_Lab_IA` avant toute modification, pour confirmer la présence des 4 commits annoncés et l'absence de modification parasite :

Commit	Contenu
df46f82	Vague V1 — <code>AGENTS.md</code> , <code>.mcp.json.example</code> , <code>handoff/</code>
9e0b203	Vague V2 — 3 nouveaux skills
5e3a44d	Vague V3 — mise à jour <code>riemann-code-review/riemann-security-review</code>
70e7463	Correctif V3 — cache Riemann-Siegel confirmé

Les 4 commits étaient bien présents, dans l'ordre attendu. Des fichiers non suivis et modifications non commitées étaient présents dans l'arbre de travail (PDF, scripts, archives), mais sans rapport avec les Vagues V1-V3 — laissés tels quels, non touchés.

3. Copie active de **Handoff.md** — arbitrage

Vérification demandée explicitement dans la consigne avant toute écriture. Lecture croisée du fichier wiki, qui indique lui-même sa propre péremption :

(!) Ce fichier était resté figé au 16 août 2026 [...] Le suivi actif de session vit en pratique dans `~/riemann_handoff/Handoff.md` (local, hors dépôt) — section « REPRISE ICI » — c'est lui qui contient le détail à jour.

Confirmé : `~/riemann_handoff/Handoff.md` (local, jamais commité) est la copie active. Le `Handoff.md` du wiki n'est qu'un pointeur court, resynchronisé ponctuellement. Mise à jour effectuée en conséquence : détail complet dans le fichier local, pointeur court propagé dans le wiki, sans dupliquer le contenu détaillé.

4. Mises à jour effectuées

`~/riemann_handoff/Handoff.md` (local, non versionné)

- Nouveau pavé daté du 27/09/2026 ajouté en tête du bloc `PROMPT_REPRISE` (résumé des Vagues V1-V3, état des 7 skills, 6 points en attente).
- Bandeau de synthèse en tête de fichier mis à jour (« Dernière mise à jour »).

`Riemann_Lab.wiki/Handoff.md`

- Section « REPENDRE ICI » remplacée par le nouveau résumé (pointeur court vers le fichier local), l'ancienne section (`MARGE_SECURITE`, 13/09) archivée sous un titre « Historique », selon le pattern déjà en usage dans ce fichier pour les sections closes.

`Riemann_Lab.wiki/JOURNAL.md`

- Nouveau pavé append-only ajouté en tête (le plus récent en haut, comme imposé par la règle du fichier), format « `## AAAA-MM-JJ - HHhMM - titre - commit` », détaillant les 3 vagues, l'état des 7 skills et les 6 points en attente.

Commit unique sur le dépôt wiki (master), poussé sur origin :

Commit	Dépôt	Contenu
7481432	<code>Riemann_Lab.wiki</code> (master)	Mise à jour <code>Handoff.md</code> + <code>JOURNAL.md</code> — session 27/09

Aucun commit ni modification sur le dépôt de code (`Riemann_Lab_IA`) pendant cette tâche — conforme à la consigne (documentation uniquement).

5. Points laissés en attente (rappel explicite du périmètre)

Conformément à la consigne, les 6 points suivants ont été documentés dans le `Handoff` mais aucune action n'a été engagée dessus :

1. Réingestion RAG BrainVault (corpus figé depuis le 25/07/2026).
2. « Cerveau autonome zêta » (vision 23 points) — chantier séparé ou intégré ?
3. Anomalie non résolue : déficit de 63 zéros entre un run `MARGE_SECURITE=10.0` et un run `MARGE=2.0`.
4. Vague V4 (tests/CI) — non lancée.
5. Vague V5 (réorganisation `phase-c-illinois/CLAUDE.md/scripts`) — non lancée.

6. Gate empirique T=100k MARGE_SECURITE (reprise dormante antérieure, toujours en attente, indépendante de ce chantier skills).
-

Partie B — Validation end-to-end du MCP OmniRoute et service systemd

Objectif : reprendre le chantier MCP `omniroute` entamé plus tôt dans la journée (trois correctifs Node/stdout/DB déjà appliqués mais non encore validés en conditions réelles), le faire fonctionner de bout en bout jusqu'à un vrai appel de chat completion, puis documenter et clôturer proprement.

6. Cadrage — reprise du test

Consigne d'ouverture, citée verbatim :

Reprend notre test `mcp`

La mémoire de session a montré que les trois causes du dysfonctionnement MCP OmniRoute (mauvaise version de Node résolue via PATH, pollution de stdout cassant le framing JSON-RPC, DB jamais initialisée au démarrage) avaient été diagnostiquées et patchées, mais jamais validées via une vraie connexion MCP de Claude Code — seulement en standalone.

7. Premier contrôle — `omniroute_get_health`

Le premier appel réel a réussi (JSON valide, aucun `CONNECTION_CLOSED`, aucune erreur DB), confirmant que les trois correctifs tiennent en conditions réelles. Un champ `degraded` restait présent (`fetch failed` sur `health/resilience/rateLimits`), d'abord interprété comme cosmétique (dashboard non lancé, sans impact sur le routage réel).

8. Test end-to-end réel — première tentative

Consigne, citée verbatim :

Ignore le `degraded` (juste le dashboard web port 20128 éteint, sans impact connu sur le routage réel). Enchaîne directement sur le test end-to-end [...]

L'appel `omniroute_route_request` a échoué avec la même erreur `fetch failed`. En traçant le code du serveur MCP (`sourceEndpoints` de l'outil pointant vers `/v1/chat/completions`), il est apparu que l'hypothèse initiale était fausse : le serveur MCP `stdio` n'a **aucune** logique de routage autonome — il délègue tout, y compris les vrais appels de chat completion, au backend HTTP local (`http://localhost:20128`). Sans ce backend démarré, aucun outil qui appelle un provider ne fonctionne, pas seulement le monitoring. Correction transmise et actée.

9. Mise en service durable du backend (`systemd --user`)

Consigne détaillée, citée verbatim (extrait) :

Le backend OmniRoute (port 20128, commande `omniroute` sans `--mcp`) doit tourner en permanence [...] Mets en place un service `systemd --user` pour le faire tourner en arrière-plan de façon durable [...] Demande confirmation avant de créer le service `systemd` si tu as un doute sur l'impact.

Avant de créer une unit manuelle, la commande intégrée `omniroute autostart enable` a été testée — question posée à hprzeta pour arbitrer entre les deux approches, tranchée en faveur du mécanisme

intégré. Résultat inattendu : sur cette machine, il choisit un mécanisme XDG desktop (`~/.config/autostart/omniroute.desktop`, ne démarre qu'en session graphique), sans option pour forcer systemd. Retour à la création manuelle, comme prévu initialement, en réutilisant la commande `Exec` exacte du fichier `.desktop` généré.

Blocage rencontré : la première tentative d'écrire directement l'unit systemd via l'outil Write de Claude Code a été refusée par le classificateur de permission du harnais (motif : action de persistance système non autorisée — création d'un service à démarrage automatique). hprzeta a créé le fichier lui-même, commande par commande, dans son propre shell.

Unit finale (`~/.config/systemd/user/omniroute.service`) :

```
[Unit]
Description=OmniRoute – routeur IA (backend HTTP, port 20128)
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
ExecStart=/home/riemann/.nvm/versions/node/v22.23.3/bin/node /home/riemann/.nvm/versions/node/v22.23.3/lib
/node_modules/omniroute/bin/omniroute.mjs serve --no-open
Restart=on-failure
RestartSec=5

[Install]
WantedBy=default.target
```

Activation confirmée : service actif, port 20128 en écoute sur 127.0.0.1, `/api/monitoring/health` → 200 `{"status":"healthy","setupComplete":true}`.

10. Test end-to-end réel — succès

Consigne, citée verbatim :

Mais avant, essaie d'abord un test simple avec un modèle du provider NVIDIA (déjà connecté avec succès, ex: `nvidia/nemotron-3-super` ou `nvidia/z-ai/glm5`) plutôt que `aimlapi/deepseek` qu'on n'a pas configuré.

Appel réussi vers `nvidia/nemotron-3-super-120b-a12b` avec la question « Quelle est la valeur de $\zeta(2)$? » : réponse mathématiquement correcte ($\zeta(2) = \pi^2/6$, problème de Bâle, Euler), 27 tokens prompt / 612 completion, latence 8.4 s. Pipeline complet validé de bout en bout : MCP studio → backend HTTP systemd → provider NVIDIA → réponse.

Résiduel non bloquant, documenté mais non traité (hors périmètre) : 403 `AUTH_001 Invalid management token` sur les outils de gestion (`get_health` détaillé, `check_quota`, `list_models_catalog`) ; provider `aimlapi/DeepSeek` non configuré (401 `No active credentials`), contourné avec NVIDIA sans nécessité de credentials supplémentaires.

11. Revue de sécurité et commit

Avant de committer la documentation (`riemann-agent-bridge/SKILL.md §1bis` et `.mcp.json.example`), le skill `riemann-security-review` a été lancé sur demande explicite. Verdict sûr à pousser : aucun secret en clair, `.mcp.json` réel toujours ignoré par Git, `.mcp.json.example` correctement suivi (gabarit sans clé).

Commit `3e352a3` sur `Riemann_Lab_IA`, poussé sur `origin/Riemann_Lab_IA`.

12. Point projet et mise à jour de la documentation de suivi

Sur la commande « **point projet** », le bloc `PROMPT_REPRISE` de `~/riemann_handoff/Handoff.md` a été affiché tel quel puis résumé. Sur confirmation explicite, ce bloc a été entièrement réécrasé (pas empilé) avec l'état du chantier OmniRoute, en tête du résumé du jour déjà présent (Vagues V1-V3 + V2bis du plan de mutualisation skills/MCP, cf. Partie A). L'en-tête du fichier a ensuite été mis à jour sur demande explicite pour refléter le nouveau dernier commit (3e352a3 au lieu de 8a2ef34).

Enfin, sur demande « **Push le wiki avec le JOURNAL.md et Handoff.md mis à jour** » : une nouvelle entrée datée a été ajoutée en tête de `JOURNAL.md` (append-only, le plus récent en haut), le pointeur « **REPRENDRE ICI** » du `Handoff.md` du wiki a été mis à jour, et les deux fichiers ont été commités et poussés (6161097 sur `origin/master`).

Incident mineur résolu au passage : un fichier `.git/index.lock` obsolète (18h08, antérieur à cette session, aucun processus ne le détenant — vérifié via `fuser` avant suppression) bloquait le dépôt wiki ; supprimé après confirmation qu'aucun processus git n'était réellement actif.

Partie C — Piège Write découvert et capitalisé en mémoire

À la clôture de la Partie B, l'archive de session a été générée une première fois avec l'outil Write, sans relire l'état existant du fichier au préalable.

13. Détection de l'écrasement silencieux

Le fichier `docs/archive/discusion_claude_2026-09-27.md` existait déjà, commité plus tôt dans la journée (88dc984, Partie A ci-dessus) — le write initial a remplacé tout son contenu par le seul récit de la Partie B, perdant la Partie A du répertoire de travail (récupérable via git, mais disparue du fichier tel que vu). Repéré avant tout commit, via `git diff --cached --stat` montrant des suppressions inattendues pour ce qui aurait dû être un simple ajout. Correction : contenu de la Partie A restauré depuis le commit 88dc984, fusionné avec la Partie B en un seul document (sections numérotées en continu), recommité (22c7898, poussé sur `origin/Riemann_Lab_IA`).

14. Capitalisation en mémoire

Consigne, citée verbatim :

ajoute une note sur ce piège Write dans la mémoire `feedback_bye_bye_archive.md`

Une section a été ajoutée à la mémoire `feedback_bye_bye_archive.md` : vérifier systématiquement, avant tout write d'archive de session, si un fichier du même jour existe déjà (`git log --oneline -- <chemin>`), et le cas échéant fusionner (sections continues, jamais un remplacement intégral) plutôt que d'écraser. Réflexe de détection ajouté : un `git diff --cached --stat` montrant des suppressions proches de la taille du fichier d'origine signale un écrasement, pas un ajout.

15. État de synchronisation en fin de journée

- `Riemann_Lab_IA` : à jour avec `origin` (22c7898 poussé, dernier en date — inclut la fusion Partie A + Partie B de cette archive).
- `Wiki master` : à jour avec `origin` (6161097 poussé).

- Backend OmniRoute : service `systemd --user` actif et persistant (`enable --now`), survivra à un redémarrage de session.
- Mémoire de session : `feedback_bye_bye_archive.md` enrichie du piège Write (§13-14 ci-dessus), `MEMORY.md` mis à jour en conséquence.
- Points restés hors périmètre, non traités : token de gestion OmniRoute invalide (`AUTH_001`), absence de credentials `aimlapi/DeepSeek`, 6 points d'arbitrage en attente sur les Vagues V4/V5 du plan skills/MCP (Partie A, §5 — inchangés).

Document créé le 27 septembre 2026 — Riemann_Lab — hprzeta.